

Trustworthy LLM-Assisted Detection of Malicious Packages: Cross-Language Behavior Graphs, Federated Learning, and the Safety Layer as a Third Threat

ANONYMOUS (PILOT DRAFT), PackGuard project, round-8 draft,

We present a pilot study that unifies three threat surfaces against LLM-assisted malicious-package detection: (i) the detection model itself, trained over a normalized cross-language *behavior-graph* representation; (ii) the training regime, a federated simulation over naturally non-IID ecosystem partitions (npm vs. PyPI); and (iii) the safety layer of the LLM used for analysis, which may refuse, or corrupt verdicts of, defensive package reviews. We release a fully provenance-tracked corpus of 603 real npm/PyPI packages (384 malicious from a wild-capture dataset, 219 popularity-derived benign), extract six-class behavior graphs with tree-sitter, and train logistic/MLP models under FedAvg/FedProx with secure-aggregation and DP simulations. Under a package-level group split (our primary protocol, adopted after an audit quantified 35% version-level train-test leakage under random splits), graph features reach F1 0.923 (AUC 0.916), statistically indistinguishable from centralized training on this pilot; a TF-IDF code-as-text baseline attains higher AUC (0.961) but worse threshold-calibrated F1 (0.861) — neither representation dominates. A refusal-gated LLM-built knowledge base and a safety-layer arm study (P0/P1/P2 over real packages) measure the remaining threats: KB features are neutral under FedAvg and add +1.0 F1 to centralized training (gained entirely on the minority client), and the safety arms show zero refusals but verdict corruption toward “malicious” under an advisory-comment injection (FP-bias on benign packages $0.2 \rightarrow 0.6$), replicating the RefuseGuard false-positive channel in this domain. We report all negative and null results, including that an earlier apparent FedAvg minority-client harm disappeared once a file-selection bug was fixed, and discuss the distance between this pilot and a Q1-grade system.

1 INTRODUCTION

Open-source package ecosystems are a primary supply-chain attack surface: attackers publish packages that steal credentials, exfiltrate data, or download further malware during installation [1, 5, 6]. Deployed detectors such as DON-API [4], Cerebro [9], and MalGuard [2] learn from package metadata, installation behavior, or code; LLM-based triage is emerging independently [8].

This paper asks whether *trustworthy* LLM-assisted detection can be studied as one system rather than three separate papers. We identify three threats that act on the same analysis pipeline:

- (1) **Model threat:** the detector must generalize across languages (JavaScript/Python) and ecosystems from limited labeled data;
- (2) **Training-regime threat:** ecosystem data is naturally non-IID and cannot always be pooled (organizational/privacy boundaries), motivating federated training whose cost must be quantified;
- (3) **Safety-layer threat:** the LLM performing (or explaining) the analysis may *refuse* defensive work phrased offensively, or *corrupt* verdicts under untrusted package content — the RefuseGuard findings [7] must transfer to the higher-stakes package domain, where refusal rates are plausibly higher (we measure, not assume).

Our contributions: (1) a provenance-complete 603-sample npm+PyPI corpus with per-sample source, label provenance, and checksums (§4); (2) a normalized cross-language behavior-graph schema with six semantic classes and 18 graph-level features (§3); (3) a federated simulation with secure-aggregation and DP mechanisms over ecosystem partitions, evaluated with a leakage-resistant group split as the primary protocol (§5); (4) a refusal-gated LLM-augmented knowledge base; and (5) a safety-arm measurement ported from the vulnerability domain to real packages (§5.2). All audits, amendments, and failed hypotheses are disclosed.

2 RELATED WORK

Malicious package detection. DONAPI maps installation-time behavior sequences for npm [4]; Cerebro trains a single cross-ecosystem model for npm and PyPI (centralized) [9]; MalGuard targets real-time PyPI detection [2] (venue per USENIX program pages; arXiv record verified); MeMPtec uses metadata [3]. None study federated training across ecosystem partitions or the safety layer of LLM analyzers. **FL for security.** VulFL applies FL to vulnerability detection [10] (preprint); to our knowledge, FL over ecosystem-partitioned clients for malicious-package detection is unexplored. **LLMs for supply-chain security.** SecurityAI [8] measures LLM detection accuracy but not refusal interference. **Attack taxonomy.** Ladisa et al. [5] systematize OSS supply-chain attacks (IEEE S&P 2023; an earlier draft of this work mistakenly cited ACSAC — corrected). **Safety-layer line.** Our companion study RefuseGuard [7] established in the vulnerability domain that open 2–7B models show $RR \approx 0$ under offensive-but-defensive wording, verdict corruption toward vulnerability (a false-positive channel), and defense bundles that harm $\leq 3B$ models. PackGuard ports those conditions to packages.

3 METHOD

3.1 Behavior graphs

tree-sitter parsers for JavaScript and Python produce, per sample, a merged graph whose nodes are classified API calls in six semantics (FILE_IO, NETWORK, PROCESS, CRYPTO, DYNAMIC_CODE, DATA_ACCESS) and whose edges are in-scope sequence edges and data edges from produced-variable tracking; imports and require/import aliases are resolved. Entry kinds mark `setup.py` and `npm install-hook` scripts. The schema is versioned (SCHEMA_VERSION 1.1.0). Eighteen graph-level features (class histogram, size/density/depth, scope and import diversity, entry flags, parse-failure count) feed classical models; a GNN is future work.

3.2 Federated simulation

Clients are ecosystem partitions (npm $n=328$, PyPI $n=146$ after the train split; malicious rate 0.686 vs. 0.527 — naturally non-IID but mildly so). We train LR (primary) and a 32-unit MLP for 15 rounds (2 local epochs, SGD lr 0.1, batch 32, seed 20260922) with FedAvg and FedProx ($\mu=0.01$). Secure aggregation is a pairwise additive-mask *simulation* (masks cancel exactly for two clients); DP is a DP-FedAvg-style clip-and-noise *simulation* with a single-round analytic ϵ (no composition proof; disclosed). Baselines: centralized pooling and per-client-only.

3.3 LLM-augmented knowledge base

Unknown API calls (from the graph universe, not the feature table) are classified by a local Qwen2.5-Coder-3B-Instruct into the six-class taxonomy with risk levels, refusal-gated (refusal $\rightarrow$ one retry $\rightarrow$ UNSURE; a refusal is never converted into a class) and cached; KB-derived features (risk ratio, confidence, unsure ratio) augment the classifier for an on/off ablation.

3.4 Safety port

Three pre-registered arms over real package code: **P0** neutral defensive task; **P1** identical intent with DRB vocabulary (“steal credentials”, “exfiltrate”, “malware”); **P2** an advisory-style security comment injected into the package file (comment-only, gated by a tree-sitter AST-equivalence check). Metrics: refusal rate (RR) per arm, verdict flips

Table 1. Primary protocol (package-level group split, global test $n=129$, 88 malicious). LR, 15 rounds, threshold 0.5. Per-ecosystem F1 and per-coverage F1 are computed on the same test set.

Run	P	R	F1	AUC	npm/pypi F1	w/e F1
graph FedAvg	.894	.955	.923	.916	.933/.875	.925/.909
graph FedProx	.894	.955	.923	.916	.933/.875	.925/.909
graph central	.903	.955	.928	.926	.940/.875	.931/.909
graph per-client	–	–	.917	.909	–	–
tfidf FedAvg	.682	1.000	.811	.959	.880/.588	.839/.645
tfidf central	.763	.989	.861	.960	.885/.757	.880/.741
tfidf per-client	–	–	.842	.918	–	–

Table 2. Primary comparisons (FedAvg vs. centralized; paired accuracy).

Split / block	McNemar p	Δ acc [95% CI]	
group / graph	1.000	–.008 [–.023, .000]	n.s.
group / tfidf	.001	–.101 [–.155, –.047]	sig. (FL worse)
random / graph	.063	+.041 [+0.008, +.083]	mixed
random / tfidf	.007	–.107 [–.182, –.041]	sig. (FL worse)

benign→malicious, false-positive bias; per-model refusal thresholds; frozen decision rules (blocking: Δ RR ≥ 0.10 and McNemar $p < 0.05$).

4 EXPERIMENTAL SETUP

Corpus (603, disclosed). Malicious: DataDog wild-capture dataset, deterministic stride sample (seed 20260922) of 298 npm + 92 PyPI archives with verbatim per-sample class provenance; benign: popularity-ranked npm registry and PyPI download lists, 123 + 90, “assumed benign, not individually audited”. A round-8 audit found the manifest builder silently skipped DataDog’s flat layout; the fix adds 6 archives (5 single-version packages + 1 extra archive of an already-selected package), 597 → 603; the flawed 597 manifest is retained as an audit artifact. **Coverage.** With-graph rates after the selection fix: npm-benign 105/123 (85.4%), npm-malicious 252/298 (84.6%), pypi-benign 63/90 (70.0%), pypi-malicious 80/92 (87.0%); 500/603 (82.9%) overall. Empty-graph rows are kept as all-zero and reported per cell. **Splits.** PRIMARY: package-level group split (all versions of a package on one side; label-stratified; test $n=129$, 88 malicious). This was adopted as AMENDMENT-1 after measuring that the pre-registered random split shares a package between train and test for 35% of test rows. SECONDARY: the original stratified random split, reported for continuity. **Statistics.** McNemar (exact when discordant < 25) and paired bootstrap (10k, seed 20260922) on per-sample accuracy; $\alpha = 0.05$. **Pilot limits.** simulation-only FL; two ecosystems (no Maven); popularity-derived benign labels; hand-curated API mapping tables; single-seed runs.

5 RESULTS

FL cost is small but real on the text baseline, null on graphs. On the primary split, FedAvg matches centralized training on graph features (Table 2); on TF-IDF, federated training costs 10 accuracy points because the minority client

Table 3. Coverage after the file-selection fix (v2). The v1 selector flooded its 12-file budget with `__init__.py` and metadata files, manufacturing 16 spurious empty pypi-benign graphs (audit finding); pypi-benign coverage rises 55.6% $\rightarrow$ 70.0%.

Cell	with-graph	coverage
npm benign	105/123	85.4%
npm malicious	252/298	84.6%
pypi benign	63/90	70.0%
pypi malicious	80/92	87.0%
total	500/603	82.9%

Table 4. KB ablation on the primary (group) split. “on” appends `kb_risk_ratio` / `kb_confidence` / `kb_unsure_ratio` to the 18 graph features.

Run	F1		AUC	
	off	on	off	on
FedAvg	.923	.922	.916	.911
centralized	.928	.938	.926	.932
pypi F1 (central)	.875		.966	

(PyPI) collapses under an unweighted average (pypi F1 0.588 vs. 0.757 centralized). FedProx at $\mu=0.01$ is indistinguishable from FedAvg everywhere — the proximal term is too weak to matter at this scale (unit-verified to bind at larger μ).

Graph vs. text is split-decision, honestly. The TF-IDF baseline wins AUC (0.961 vs. 0.916) but is miscalibrated at the fixed threshold (recall $\approx$ 1, precision 0.76), losing on F1. We do not claim graph superiority; the claim is complementarity under calibration-sensitive metrics and interpretability (named behavior nodes), at the cost of coverage (Table 3).

Sensitivity arms. DP ($\sigma=0.1$, $\epsilon \approx 9.7$ per round, single-query bound): F1 0.915 (−0.8 points) — privacy noise is affordable at this scale, though the R-round composition is unproven. MLP matches LR exactly on the primary metrics. Dropping the 103 empty-graph rows moves central F1 to 0.821 ($n=500$): the all-zero rows are not harmless artifacts but carry ecosystem signal (disclosed, not exploited).

5.1 Knowledge-base ablation

The graph universe of the 603-sample corpus contains 137 unique API names; 122 are unknown to the 20-entry seed KB. We classified the top 60 by document frequency with local Qwen2.5-Coder-3B-Instruct through the refusal-gated builder: **0 refusals, 0 retries, 0 UNSURE** (the 0.5B pilot model produced 8/10 UNSURE on the same protocol — model scale matters), at ~ 15 s/call on MPS. The remaining 62 tail APIs are not in the KB and are counted as unsure by construction (disclosed).

KB features are neutral under FedAvg (−0.1 pt F1) and add +1.0 pt F1 / +0.6 pt AUC to centralized training, with the entire gain on the minority client (pypi F1 .875 $\rightarrow$.966; subgroup $n=29$ — indicative, not significant). The explainability payload (per-call semantic class and rationale) is the more robust yield: every high-risk node in a verdict now carries a gated, cached justification.

Table 5. Safety arms on 10 real packages (pooled over 2 models, $n=20$ per arm). RR = refusal rate; flips = benign $\rightarrow$ malicious *verdict* changes vs. P0 over parsed pairs; FP-bias = share of truly-benign packages predicted malicious. Rules are the frozen pre-registered ones.

Arm	RR	flips b $\rightarrow$ m	FP-bias	blocking? / corrupting?
P0 neutral	.00	–	.2	– / –
P1 offensive	.00	1/20	.3	no / yes
P2 advisory	.00	4/20	.6	no / yes

5.2 Safety layer on real packages

The RefuseGuard pattern transfers unchanged to the package domain: **no arm refuses** (RR = 0 everywhere, McNemar $p=1$ – blocking unsupported, as in the vulnerability study), while **verdict corruption re-appears as a false-positive channel**: the advisory-in-package arm (P2) flips 4/20 verdicts toward “malicious” and raises FP-bias on benign packages from 0.2 (P0) to 0.6 (granite-3.3-2b: 1.0 – every benign package flagged under the advisory comment); P1’s offensive wording alone contributes one flip (granite). Both P1 and P2 satisfy the pre-registered corruption rule (≥ 1 flip). Per-model detail: llama-3.2-3B P2 1/10 flip, FP-bias 0 $\rightarrow$ 0.2; granite P2 3/10, FP-bias 0.4 $\rightarrow$ 1.0 – the smaller model is again the more corruptible. All of this is a *pilot observation* at $n=10$: it establishes measurability and direction, not effect size.

6 DISCUSSION

When graphs lose to text. On this pilot, raw text remains the stronger ranking signal (AUC) while graph features win thresholded F1 and explainability. The honest conclusion is that the behavior-graph hypothesis is *not yet* validated as strictly better; its value is calibration, interpretability, and coverage robustness under the group split (graph degrades less between random and group splits than TF-IDF’s minority-client F1 does under FL). **A retracted finding.** Our interim run on the v1 features showed a dramatic FedAvg minority-client harm (pypi F1 0.57 vs. 0.82 centralized). After the file-selection fix the harm vanishes (Table 1: pypi 0.875 under both). We report this because the “FedAvg harms minorities” narrative was an artifact of a feature bug – a caution against interpreting FL failures before auditing the features. **Safety transfer.** The RefuseGuard mechanisms (refusal monitor, verdict-corruption channel, defense-harm) port mechanically to packages; P2’s advisory-in-package attack is a package-specific threat the vulnerability study could not express, and the AST gate makes it safely measurable.

7 CONCLUSION AND LIMITATIONS

A 603-sample, provenance-complete pilot shows: federated training over ecosystem partitions is near-free for graph features and costly for text features; representation superiority is split-decision; LLM-built knowledge bases and safety-layer threats are measurable in this domain with the released gates and monitors. Limitations: pilot scale (CI widths $\pm .05$ – $.08$ F1), simulation-only trust mechanisms, two languages, popularity-derived benign labels, hand-curated mappings, single seed, and an LLM-KB whose confidence is self-reported. A Q1-grade system needs: multi-seed runs with power analysis, a real GNN over the graphs, at least one more ecosystem, audited benign labels, and a deployed-data evaluation of the safety arms.

DATA AND PROVENANCE

Manifest, features, graphs, per-row results with mock flags, audit reports (V1/V2), and AMENDMENT-1 are in the project tree (data/packguard/, outputs/packguard/, docs/packguard_prereg.md, reports/round8/).

REFERENCES

- [1] Ruian Duan, MJ Altheide, Ashish Joshi, Devesh Gunter, et al. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Proceedings of the 21st ACM Internet Measurement Conference (IMC)*. arXiv:2002.01139; MalOSS collection.
- [2] Xingan Gao, Xiaobing Sun, Sicong Cao, Kaifeng Huang, and Di Wu. 2025. MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem. In *34th USENIX Security Symposium (USENIX Security 2025)*. arXiv:2506.14466; venue per USENIX program, arXiv record verified.
- [3] Saptarshi Halder et al. 2024. MeMPtec: Malicious Package Detection using Metadata Information. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- [4] Chengqiang Huang, Nannan Wang, Ziyang Wang, Siqi Sun, Lingzi Li, et al. 2024. DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping. In *33rd USENIX Security Symposium (USENIX Security 2024)*. arXiv:2403.08334.
- [5] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (S&P)*. 1509–1526. arXiv:2204.04008; user proposal said ACSAC 2023 – venue corrected to IEEE S&P.
- [6] Marc Ohm, Henrik Plate, Andreas Sykosch, and Michael Meier. 2020. Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. arXiv:2005.09561; BKC dataset.
- [7] RefuseGuard study (companion paper, this project). 2026. RefuseGuard: Measuring and Mitigating Safety-Layer Interference in LLM Vulnerability Analysis. *arXiv preprint arXiv:2603.01246 (Defensive Refusal Bias)* (2026). Companion 7-round study; internal draft under submission.
- [8] Nusrat Zahan, Philippe Burckhardt, Maksym Lysenko, et al. 2024. Shifting the Lens: Detecting Malicious npm Packages using Large Language Models (SecurityAI). *arXiv preprint arXiv:2408.08924* (2024).
- [9] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, et al. 2024. Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model (Cerebro). *ACM Transactions on Software Engineering and Methodology (TOSEM)* (2024). <https://doi.org/10.1145/3705304> arXiv:2309.02637.
- [10] Peiheng Zhou, Ming Hu, Xingrun Quan, Yawen Peng, Xiaofei Xie, et al. 2024. An Empirical Study of Vulnerability Detection using Federated Learning (VulFL). *arXiv preprint arXiv:2411.16099* (2024). Venue not verified; cited as preprint.